

Overview

口腔機能・栄養評価

配布・運用の全体像

Generated from README.md / 2026-05-19 00:49

PDF 化するときは A4 でそのまま印刷できます。見出し、図、コードブロックが崩れにくく、改ページの空きが大きくなりすぎないように調整しています。

口腔機能・栄養評価システムを Synology NAS / Docker で自己ホストし、Tailscale Serve で HTTPS 終端するための構成です。

このフォルダの [index.html](#) は Claude artifact の wrapper ですが、[server.py](#) がその中に埋め込まれた本体 HTML を抽出し、保存処理だけを SQLite API に差し替えて配信します。これにより、既存レイアウトをほぼそのまま維持しつつ、複数端末で記録を共有できます。

関連資料

- Synology + Tailscale 配置手順: [DEPLOY_SYNOLOGY_JA.md](#)
- 非エンジニア向け index.html 更新手順: [GITHUB_BROWSER_UPDATE_GUIDE_JA.md](#)
- 言語聴覚士向け Claude 画面修正手順: [ST_CLAUDE_UPDATE_GUIDE_JA.md](#)
- GitHub Actions 自動反映手順: [AUTO_DEPLOY_GITHUB_ACTIONS_JA.md](#)
- Windows 利用者向け Tailscale 接続ガイド: [TAILSCALE_CLIENT_GUIDE_JA.md](#)
- タブレット利用者向け Tailscale 接続ガイド: [TAILSCALE_TABLET_GUIDE_JA.md](#)
- タブレット利用者向け案内文テンプレート:
[TAILSCALE_TABLET_MESSAGE_TEMPLATE_JA.md](#)
- タブレット利用者向け QR 案内シート: [TAILSCALE_TABLET_QR_SHEET_JA.md](#)
- 初心者向け運用マニュアル: [OPERATIONS_MANUAL_JA.md](#)
- Windows 用かんたん起動: [TailscaleClientLauncher.cmd](#)
- 起動ツール設定: [TailscaleClientLauncher.settings.json](#)
- 実運用設定のひな形: [.env.example](#)
- 現在の実運用設定: Git 管理対象外の [.env](#) を各環境で配置して使います。

想定構成

1. アプリ本体は Synology Container Manager 上の Docker コンテナで動かします。
2. `compose.yaml` はホストの `127.0.0.1:8010` にだけ公開し、LAN から直接見えないようにします。
3. Synology 上の Tailscale Serve が `https://diskstation.tail632bc4.ts.net/` を終端し、ローカルの `http://127.0.0.1:8010` へ流します。
4. 利用者は Tailscale に接続したうえで、その HTTPS URL からアクセスします。

DSM のリバースプロキシや独自ドメイン、ポート開放は前提にしています。

認証とアクセス制御

既定では認証を有効にして起動します。Tailscale 運用向けの既定値は `tailscale-or-password` です。

主な環境変数:

- `KOUKU_KINOU_AUTH_MODE` : `password` / `tailscale` / `tailscale-or-password`
- `KOUKU_KINOU_PASSWORD` : 管理者用パスワード。`tailscale-or-password` の予備入口として使います。
- `KOUKU_KINOU_SESSION_TTL_MINUTES` : パスワードログイン時のセッション有効時間。既定 480 分
- `KOUKU_KINOU_SECURE_COOKIE` : `1` で Secure Cookie を有効化。Tailscale HTTPS では `1` を維持します。
- `KOUKU_KINOU_ALLOWED_NETWORKS` : 追加の IP 制限が必要な場合だけ使います。
- `KOUKU_KINOU_TRUST_PROXY` : `X-Forwarded-For` ベースの制御が必要なときだけ `1` にします。

`tailscale-or-password` では、Tailscale の HTTPS URL から来た利用者はヘッダーで自動認証されます。Tailscale が使えない場合だけ、管理者用パスワードでの入口が残ります。

ローカル起動

`.env` がある場合は、`python server.py` 実行時にも自動で読み込みます。

PowerShell 例:

```
python server.py --auth-mode password --host 127.0.0.1 --port 8010
```

認証を切って画面確認だけしたい場合は以下です。

```
python server.py --no-auth --host 127.0.0.1 --port 8010
```

起動後に以下へアクセスします。

- <http://localhost:8010/>
- ヘルスチェック: <http://localhost:8010/api/health>

SQLite の保存先は既定で `data/records.db` です。環境変数 `KOUKU_KINOU_DB` で変更できます。

保存ルール:

- 利用者は `氏名 + 生年月日` で識別します
- 同じ利用者の同じ評価日は新規追加ではなく上書き更新します
- 同じ利用者でも評価日が違えば履歴として別記録を残します
- 記録一覧では氏名、ふりがな、生年月日、評価日で検索できます

確認用サンプル投入:

```
python seed_sample_records.py
```

既存データを消して入れ直す場合は `python seed_sample_records.py --replace` を使います。

Claude 生成 index.html を受け取ったときに管理者がやること

今後は、言語聴覚士が Claude で新しい `index.html` を作り、管理者が検証して本番へ入れる運用を前提にします。

管理者は次の順番だけ守ってください。

1. 受け取った HTML は、いきなりリポジトリ直下の `index.html` に上書きしない
2. まず別名のファイルとして保存する
3. PowerShell で検証コマンドを実行する

4. `OK (managed)` または `OK (legacy-source)` が出たときだけ、本番用の `index.html` と差し替える
5. GitHub へ反映し、Actions と本番画面を確認する

PowerShell 例:

```
& "C:\Users\user\AppData\Local\Microsoft\WindowsApps\python.exe" .\server.py --validate-client-html "C:\path\to\index_new.html"
```

判定ルール:

- `Client HTML validation: OK (managed)`: Claude 生成済みの完成版 HTML として受け入れてよい
- `Client HTML validation: OK (legacy-source)`: 旧来の変換前 HTML として受け入れてよい
- `Client HTML validation failed: ...`: その HTML は置き換えない。作成者へ差し戻す

受け入れてはいけないもの:

- `index.html` の一部分だけ
- 途中が省略された HTML
- `server.py` も直す前提の提案
- 検証コマンドで失敗した HTML

本番反映までの最後の流れ:

1. 検証に通ったファイルでリポジトリ直下の `index.html` を置き換える
2. `main` に commit するか Pull Request を `main` へ merge する
3. GitHub Actions の `Deploy to Synology` 成功を確認する
4. 本番 URL を開き、ログイン、保存、記録一覧の表示を確認する

Docker / Synology

`.env` は作成済みです。まず中身を確認し、必要なら管理者用パスワードを変更してください。

```
KOUKU_KINOU_AUTH_MODE=tailscale-or-password
KOUKU_KINOU_PASSWORD=change-this-admin-password
KOUKU_KINOU_SESSION_TTL_MINUTES=480
KOUKU_KINOU_SECURE_COOKIE=1
KOUKU_KINOU_ALLOWED_NETWORKS=
KOUKU_KINOU_TRUST_PROXY=0
```

コンテナ起動:

```
docker compose up --build -d
docker compose ps
```

`docker compose ps` の `STATUS` で `healthy` になれば、`/api/health` まで含めて通っています。

現在の公開 URL は <https://diskstation.tail632bc4.ts.net/> です。

担当者が GitHub のブラウザから [index.html](#) を上書きして本番へ反映する手順は、`GITHUB_BROWSER_UPDATE_GUIDE_JA.md` を参照してください。構成の技術詳細は `AUTO_DEPLOY_GITHUB_ACTIONS_JA.md` にあります。

Synology 側の SSH 鍵設定をやり直す必要があるときは、Windows から `setup_synology_actions_ssh.cmd` を再実行してください。

GitHub Actions の Variables / SSH secrets を再登録したいときは、Windows から `setup_github_actions_deploy.cmd` を実行してください。

Synology で Docker 実行に root 権限が必要な場合は、先に `setup_synology_actions_ssh.cmd` を `-RegisterRootKey` 付きで実行し、その後 `setup_github_actions_deploy.cmd` を `-User root` 付きで実行してください。

Synology では DSM で SSH を有効にしたうえで NAS に入り、root 権限で Tailscale Serve を設定します。

Tailscale Serve 設定:

```
tailscale serve --bg 8010
tailscale serve status
```

`tailscale` コマンドが見つからない場合は、Synology のパッケージ実体を直接呼びます。

```
/var/packages/Tailscale/target/bin/tailscale serve --bg 8010  
/var/packages/Tailscale/target/bin/tailscale serve status
```

`tailscale serve --bg 8010` は、Tailscale の HTTPS URL から `http://127.0.0.1:8010` へ転送します。初回は HTTPS 有効化の承認画面が開くことがあります。

利用者向け配布物

Windows 利用者には、次の 3 ファイルをまとめて配布できます。

1. `TailscaleClientLauncher.cmd`
2. `TailscaleClientLauncher.ps1`
3. `TailscaleClientLauncher.settings.json`

配布用の `TailscaleClientLauncher.settings.json` には

`https://diskstation.tail632bc4.ts.net/` を設定済みです。利用者はコマンド入力なしで Tailscale 導入とアプリ起動を進められます。

タブレット利用者には、ファイル配布より次の 2 つだけを案内するほうが簡単です。

1. Tailscale の導入とサインイン方法
2. アプリ URL `https://diskstation.tail632bc4.ts.net/`

タブレット向けの詳しい案内は `TAILSCALE_TABLET_GUIDE_JA.md` にまとめています。配布時にそのまま使う文面は `TAILSCALE_TABLET_MESSAGE_TEMPLATE_JA.md`、紙配布用の QR 案内は `TAILSCALE_TABLET_QR_SHEET_JA.md` を使えます。

実装メモ

- 画面配信時に `localStorage` を使う元コードを API 呼び出しへ変換しています。
- 記録は SQLite に JSON として保存されます。
- 利用者識別は `氏名 + 生年月日`、評価識別は `氏名 + 生年月日 + 評価日` です。
- 同一利用者・同一評価日の保存は上書き、別日の評価は履歴追加です。
- 記録一覧には検索欄があり、氏名、ふりがな、生年月日、評価日で絞り込みできます。
- 共有対象は同じ NAS 上の単一 DB なので、PC とタブレットで同じ記録一覧を参照できます。

- `compose.yaml` は loopback のみに bind するため、Tailscale Serve を通らない直接アクセスを減らせます。
- `compose.yaml` の healthcheck は `/api/health` で DB と画面テンプレートの準備完了を確認します。
- `tailscale` または `tailscale-or-password` では、Tailscale identity header を使った認証に対応しています。
- `index.html` 自体は元 artifact のソースとして残しています。

Source: README.md / Generated by build_docs.py